British Auction in an RFQ System

High-level design and database schema

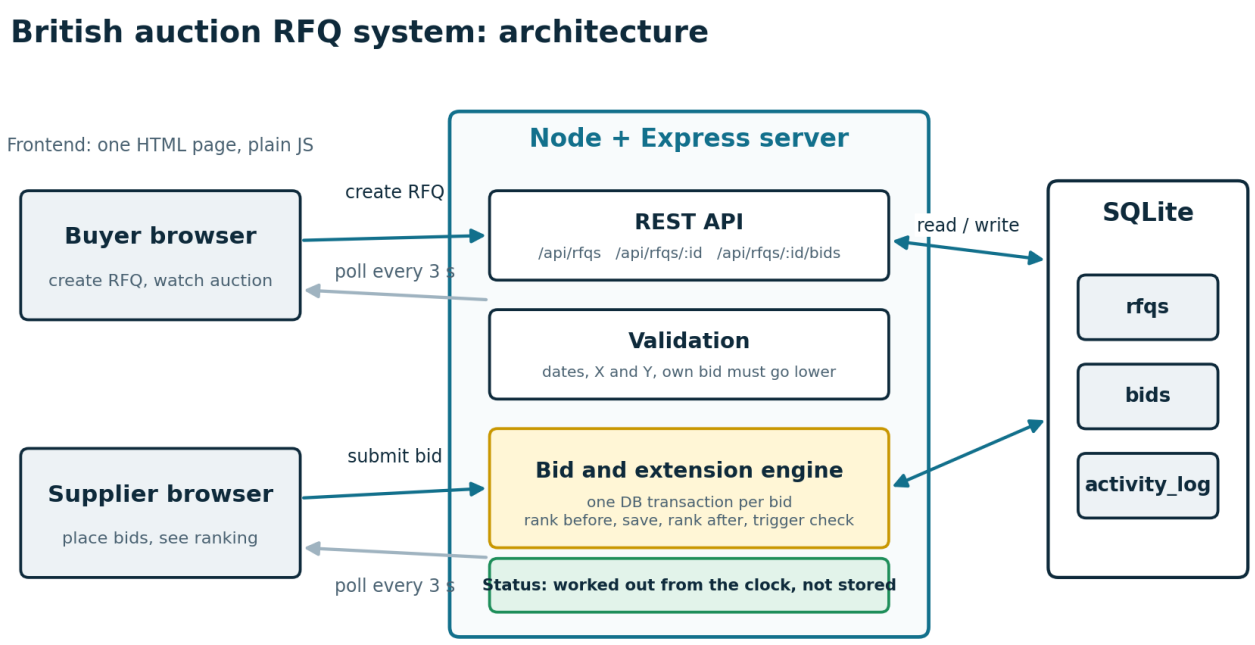
What this is

A buyer posts an RFQ (a request for quotes) and carriers bid against each other, each trying to undercut the rest. If bidding gets busy near the end, the auction stretches itself so nobody can win with a last-second bid. There is also a forced close time. Once that passes, bidding stops, no matter what.

The buyer sets three things when creating an RFQ: a watch window **X** (minutes before close), an extension **Y** (minutes added), and a trigger that decides what counts as activity: any bid, any change in supplier ranking, or a change of the lowest bidder (L1).

Stack: Node.js and Express for the backend, SQLite for storage, and a single HTML page with plain JavaScript for the frontend. The server also serves the frontend, so there is only one thing to run.

1. Architecture



The browser only talks to the REST API. All the auction rules live on the server, so the frontend can't bend them. The frontend checks for updates every 3 seconds and its countdown is corrected using the server's clock.

2. What happens when a bid comes in

1. The supplier submits a bid. The server checks that the auction is Active (started, and not yet closed) and that the numbers are valid.
2. If this supplier has bid before, the new total must be lower than their own previous total.
3. Inside one database transaction: take the supplier ranking, save the bid, take the ranking again.
4. If the bid arrived within the last X minutes before the current close, check the trigger the buyer picked.
5. If the trigger fires, the new close time is the smaller of (current close + Y minutes) and the forced close. The old and new times and the reason go into the activity log.
6. If the close is already at the forced close, nothing moves, and the log says the extension was skipped.

The three triggers

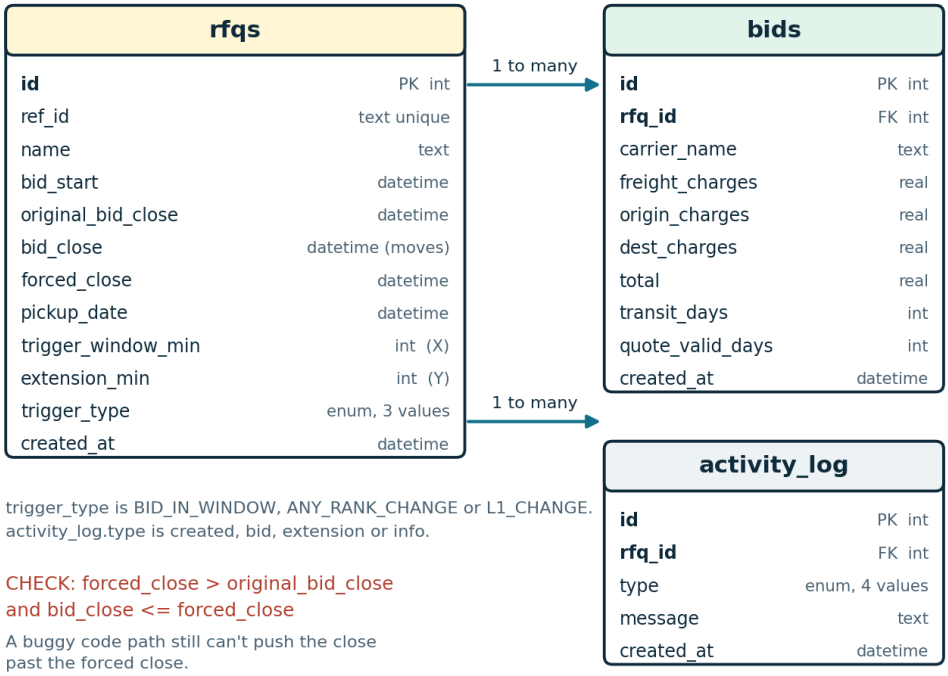
Trigger	Extends the auction when	Example
Any bid in window	Any bid arrives in the last X minutes	A bids at 5:55 with X = 10, so the close moves by Y
Any rank change	The order of suppliers changes in the last X minutes	B underbids A at 5:58, so ranks swap
L1 change	A new supplier becomes the lowest bidder	C takes L1 at 5:59, but a swap between L2 and L3 does nothing

Design decisions

- **One transaction per bid.** Two bids arriving together can't both read an old close time and overwrite each other. SQLite runs the transactions one at a time.
- **Status is worked out, not stored.** Upcoming, Active, Closed and Force Closed come from the current time and the close times. No background job that can fall behind. Force Closed means the close time was pushed all the way to the forced close and then ran out.
- **Server time only.** The browser countdown is corrected by the server's clock, so changing a laptop clock gains nothing.
- **Each supplier is ranked on their lowest bid.** A tie goes to whoever bid first.
- **Polling instead of websockets.** Simpler, and fast enough for this. Switching later would only touch the frontend and one route.
- **The database guards the main rule too.** CHECK constraints stop the close time from ever passing the forced close, even if the code has a bug.
- **Edge cases.** The very first bid counts as an L1 change, since there was no L1 before. A new supplier landing at the bottom isn't a rank change, but one landing above others is.

3. Database schema

Database schema



One RFQ has many bids and many activity log rows. `bid_close` is the live close time and moves when the auction extends; `original_bid_close` keeps what the buyer first set, so the page can show how far it was extended. `total` is freight plus origin plus destination charges, stored so ranking is a simple sort. The full SQL is in `backend/schema.sql`.

4. API

Method	Path	What it does
GET	/api/rfqs	Listing: name, lowest bid, current close, forced close, status
POST	/api/rfqs	Create an RFQ. Rejects a forced close that is not later than the bid close
GET	/api/rfqs/:id	Details: config (X, Y, trigger), ranking, all bids, activity log
POST	/api/rfqs/:id/bids	Submit a bid. This is where extensions happen

5. What was tested

Run against the real server with an automated script, and the extension screen checked by hand in the browser.

Check	Result
Forced close earlier than or equal to bid close	Rejected: Forced close time must be later than bid close time
Unknown trigger type, duplicate reference ID	Rejected with a clear message
Bid inside the window, with the extension past the forced close	Close time capped exactly at the forced close
Another bid after the cap is reached	Bid accepted, extension skipped, and logged
Bid that is not lower than the supplier's own earlier bid	Rejected, with the current total shown

Check	Result
After the forced close passes	Status is Force Closed, new bids are rejected
Rank-change trigger in the browser	B overtakes A inside the window, close moved 5:40:10 to 5:42:10 PM, reason logged

6. Not included

- No login. The carrier name is typed in, and buyer and supplier are not separate roles.
- No stats page for the success metrics (bids per RFQ, price versus non-extended auctions). The data to compute them is already in the bids table.
- Single server. A multi-server setup would swap SQLite for Postgres and keep the same transaction logic.